



SQL

03

“Structured Query Language” (Lenguaje de Consulta Estructurado) o SQL efectúa consultas para recuperar información de bases de datos así como realizar cambios en ellas.

- **Lenguaje de definición de datos** (DDL – Data Definition Language): Crear, modificar y borrar tablas.
- **Lenguaje de manipulación de datos** (DML – Data Manipulation Language): Crear, modificar y borrar registros.
- **Lenguaje de control de datos** (DCL – Data Control Language): Consultar y modificar permisos de usuarios sobre recursos (como tablas).
- **Lenguaje de consulta de datos** (DQL – Data Query Language): Realizar consultas de mayor o menor complejidad para recuperar la totalidad o parte de los datos de una o más tablas.
- **Lenguaje de control de transacciones** (TCL – Transaction Control Language): Controlar la realización de transacciones correctamente de manera que cumplan con las propiedades ACID.



SQL nació a partir del desarrollo del modelo relacional en los años 70, como “secuela” de su predecesor, SEQUEL, que funcionaba de manera similar. Fue presentado como parte de un producto comercial por primera vez en 1979 de la mano de Oracle, y desde entonces ha recibido varias revisiones, incluyendo una de las más recientes las funciones de ventana.

Numéricos

- **INT**: Almacena enteros con un rango de $(-2^{31}, 2^{31}-1)$.
- **BIGINT**: Almacena enteros con un rango de $(-2^{63}, 2^{63}-1)$.
- **SMALLINT**: Almacena enteros con un rango de $(-2^{15}, 2^{15}-1)$ (-32768, 32767)
- **TINYINT**: Almacena enteros con un rango de (0, 255).
- **DECIMAL** y **NUMERIC**: Almacenan valores numéricos de precisión exacta.
- **FLOAT** y **REAL**: Almacenan valores numéricos de precisión aproximada (“coma flotante”).

Cadena

- **CHAR**: Cadena de caracteres de longitud fija entre 1 y 255 caracteres.
- **VARCHAR**: Cadena de caracteres de longitud variable de HASTA 255 caracteres.
- **TEXT**: Cadena de caracteres de longitud variable de HASTA 63535 caracteres.
- **ENUM**: Lista de valores predefinidos con longitud fija, determinados con anterioridad (YES/NO, TRUE/FALSE, GOOD/MID/BAD, etc).
- **SET**: Lista de valores predefinidos con longitud variable.

Otros

- **BINARY**: Almacena valores binarios de hasta 8,000 bytes (8 KB)
- **VARBINARY**: Igual que el anterior pero almacenados de forma comprimida.
- **IMAGE**: Similar a VARBINARY, pero con capacidad de hasta 2 GB.
- **TIMESTAMP**: Almacena una marca de tiempo, un dato binario único. Se actualiza automáticamente cada vez que se actualiza el registro.
- **BOOL**: Valor binario: 0 (falso) o 1 (verdadero).
- **DATE**: Fecha en formato YYYY-MM-DD.



```
SELECT [[DISTINCT] columna[, columna, ...] | *]  
FROM [bd.]tabla  
[WHERE condición [AND|OR [NOT] otra_condición]]
```

- **SELECT**: Cláusula principal de una consulta. Después, podemos seleccionar columnas específicas (SELECT columna1, columna2, ...) o seleccionar todas las columnas de una tabla (SELECT *). El prefijo **DISTINCT** se salta los duplicados.
- A continuación hay que ver la procedencia de los datos. Si antes has usado **USE bd** (con el nombre de la base de datos o esquema que vas a usar en el lugar de bd) puedes quitar el prefijo “bd” en el **FROM**, pero siempre tendrás que poner al menos la tabla de la que sacas los datos.
- Lo más normal suele ser adjuntar una condición que filtre los datos para que no aparezcan todos, sino solo unos cuantos acorde a una condición. Para eso se usa **WHERE**. Puedes encadenar varias condiciones poniendo entre una y otra los operadores **AND** u **OR**, dependiendo de si quieres que se apliquen ambas o te vale con una sola. También puedes negar la condición precediéndola de un **NOT**.



```
SELECT *  
FROM productos  
WHERE precio >=100  
AND precio <= 200;
```

En el ejemplo anterior vemos un ejemplo en el que seleccionas todos los datos de la tabla productos cuyo precio esté entre 100 y 200 (sea mayor o igual a 100 y menor o igual a 200).

Condiciones:

- **Numéricas:** mayor que, menor que, igual, etc. (>, >=, <, <=, =, !=, BETWEEN a AND b, [NOT] IN (conjunto,de,datos), IS [NOT] NULL, y varios operadores aritméticos según sean necesario. IN y NOT IN operan con conjuntos, de manera que la condición se cumple si el dato está incluido en dicho conjunto.
- **De cadena de caracteres:** LIKE (con expresiones regulares), =, !=, %, [NOT] IN, IS [NOT] NULL, etc.

También hay operaciones específicas de cadenas, como CONCAT, UPPER, LOWER; etc. Que operan sobre las propias cadenas, concatenándolas, pasándolas a mayúsculas, minúsculas, etc. Por ejemplo, aquí un alumno llamado Carlos, de apellido García, pasaría a llamarse “García, Carlos”: (El **AS** nos sirve para darle nombre a la columna)



```
SELECT LOWER(CONCAT(apellido, ' ', nombre)) AS nombre_completo  
FROM alumno;
```

SQL: ORDER BY

```
SELECT [columna(s) | *]
```

```
FROM [bd.]tabla
```

```
ORDER BY columna [ASC|DESC]
```

- **ORDER BY** permite ordenar por una o más columnas de forma ascendente (por defecto) o descendente, independientemente de su tipo de dato. En caso de cadena, ordenará alfabéticamente de A a Z si es ascendente, y de Z a A en caso descendente.
- Al encadenar varias columnas, la primera tomará prioridad, y si dos registros son iguales en esa columna de ordenación pasarán a ordenarse por la segunda, y así sucesivamente.



```
SELECT [columnaX, columnaY | *], FUNC(columna|*)  
FROM [bd.]tabla  
GROUP BY columnaX, columnaY
```

GROUP BY permite agrupar los resultados por columnas determinadas. Este necesita de una función de agrupación (FUNC) como las siguientes:

- **COUNT()** – cuenta el número de filas. “*” cuenta todas las filas dada una condición (WHERE), mientras que COUNT(columna) cuenta todos los valores específicos de esa columna. Puede emplearse COUNT DISTINCT para contar solo los distintos valores de dicha columna.
- **SUM(col)** – Suma los valores de la columna dada.
- **AVG(col)** – Calcula la media de la columna dada.
- **MAX(col)** y **MIN(col)** – Devuelve los valores máximo y mínimo de la columna dada.

Ejemplo: el alumno más alto de cada clase.

```
SELECT nombre, apellidos, clase, MAX(altura)  
AS altura_max  
FROM alumnos  
GROUP BY clase  
ORDER BY MAX(altura) -- al ordenar  
debemos repetir el comando de agrupación
```



SQL: HAVING

```
SELECT [columnaX, columnaY | *], FUNC(columna|*)  
FROM [bd.]tabla  
GROUP BY columnaX, columnaY  
HAVING FUNC(columna|*) condición
```

Frente a **WHERE**, que filtra los resultados ANTES de realizar la agrupación, **HAVING** filtra los resultados una vez agrupados, siendo escrito justo después que el GROUP BY (y antes del ORDER BY).

```
SELECT nombre, apellidos, clase, MAX(altura) AS altura_max  
FROM alumnos  
GROUP BY clase  
HAVING MAX(altura) > 175 -- para ver solo las clases donde el alumno más alto mida más  
de 175 centímetros.
```




```
SELECT columnas  
FROM tablaA a  
JOIN tablaB b  
ON a.clave = b.clave
```

JOIN nos permite trabajar con datos de dos tablas a la vez (vuelven a ser relevantes claves primarias y foráneas). Hay varios tipos:

- **INNER JOIN**: Nos dará solo los datos que coinciden en ambas tablas. Si unimos clientes y pedidos, recibiremos información de registros que contengan datos de clientes Y pedidos.
- **LEFT/RIGHT JOIN**: Nos dará todas las filas de la tabla izquierda/derecha, incluyendo datos de la otra tabla donde la clave coincida. En el caso anterior dará los datos de todos los clientes y también los pedidos que hayan hecho, pero aunque no hayan hecho ningún pedido también aparecerán.
- **FULL (OUTER) JOIN**: Similar, incluye las filas de ambas tablas haya coincidencia o no, combinándolas donde coincidan y mostrándolo vacío donde no.
- **CROSS JOIN**: Combina todas las filas de la primera tabla con todas las filas de la segunda coincidan o no. Útil para el producto cartesiano (ver todas las combinaciones posibles).



Existe también el **SELF JOIN**, que consiste en unir a una tabla consigo misma mediante un JOIN estándar.

SQL: UNION, INTERSECT y EXCEPT

```
SELECT * FROM tabla1
```

```
UNION / INTERSECT / EXCEPT
```

```
SELECT * FROM tabla2
```

```
ORDER BY columna -- criterio que se aplica al resultado, no a la segunda parte
```

Tres operaciones de conjuntos que trabajan uniendo en vertical, no en horizontal:

- **UNION (ALL)**: Junta en una misma tabla los registros de la primera consulta con los de la segunda, siempre que ambas tengan el mismo esquema (o mismo número de columnas). ALL une también los duplicados, mientras que si no se incluye se los salta.
- **INTERSECT**: Junta en una misma tabla solo los registros que aparezcan íntegros en ambas consultas a la vez.
- **EXCEPT**: Junta en una misma tabla todos los registros de la primera consulta QUE NO aparezcan en la segunda consulta (funciona como un menos aritmético)



Las subconsultas deben seguir una serie de reglas:

- Una subconsulta ha de colocarse siempre en paralelo.
- Una subconsulta debe colocarse a la derecha del operador de comparación.
- Las subconsultas no pueden manipular sus datos internos, por lo que no pueden incluir ORDER BY (que sí puede incluirse en la consulta principal).
- Para subconsultas que devuelvan una sola línea deben usarse operadores de una sola línea (=, por ejemplo)
- En caso de que una subconsulta devuelva un valor nulo, la consulta externa no devolverá ninguna fila con ciertos operadores de comparación.
- Podemos usar GROUP BY en una subconsulta.
- No podemos usar BETWEEN en la consulta externa con una subconsulta, pero sí dentro de la subconsulta.
- Pueden anidarse subconsultas dentro de subconsultas, siempre que cumplan con el resto de reglas.



SQL: Subconsulta SELECT AS FIELD

```
SELECT nombre, precio  
FROM producto  
WHERE precio > (  
    SELECT AVG(precio)  
    FROM producto  
)
```

Esta consulta devuelve los productos cuyo precio es superior a la media.

Donde en el operador (“precio > ()”) puede ir cualquier tipo de comparador, siempre que coincida con el resultado que da la subconsulta, como hemos visto en las reglas anteriores (si va a devolver más de un registro, solo podríamos usar IN, por ejemplo; o EXISTS (que devuelve un booleano si la subconsulta devuelve algo))



SQL: Subconsulta SELECT FROM SELECT y SELECT SELECT FROM (nombre por determinar)

```
SELECT id, producto, MAX(total_vendido) AS mayor
FROM (
    SELECT P.id, P.nombre as producto, (
        SELECT COUNT (VP.id_producto)
        FROM venta_producto VP
        WHERE P.id = VP.id_producto -- un JOIN sin usar JOIN
        GROUP BY id
    ) as total_vendido
    FROM producto P
    GROUP BY P.id, P.nombre
) AS tabla
GROUP BY id, producto
```

Aquí vemos una consulta que consiste en ver todos los productos, cuántos se vendieron y el que tuvo la mayor cantidad de artículos vendidos. Vemos un ejemplo de anidación de subconsultas.



SQL: CREATE TABLE

```
CREATE TABLE [IF NOT EXISTS] tabla (  
    columna1 TIPO_DATO,  
    columna2 TIPO_DATO,  
    ...  
)
```

Empezamos bloque DDL y DML con la creación de tablas.

Cada columna que se añada debe incluir su nombre y el tipo de dato con las especificaciones necesarias (por ejemplo, **VARCHAR** debe incluir su longitud entre paréntesis (**nombre VARCHAR(20)**, por ejemplo).



SQL: Restricciones en creación de tablas

Las restricciones pueden escribirse en línea y en bloque. He aquí las más frecuentes:

- **NOT NULL:** Un campo no puede ser nulo.
- **UNIQUE:** El valor de un campo no puede repetirse.
- **PRIMARY KEY:** Este campo o campos forma(n) la clave primaria de la tabla.
- **FOREIGN KEY / REFERENCES:** Este campo es clave foránea y hace referencia a la tabla de la que viene.
- **DEFAULT:** Se especifica el valor por defecto que tomará este campo en caso de no quedar determinado.
- **CHECK:** Esta columna debe cumplir una condición concreta que tiene que ver con su valor.
- **CREATE INDEX:** Permite la creación de índices sobre la columna.

```
CREATE TABLE ejemplo (  
    columna1 VARCHAR(20) NOT NULL,  
    columna2 VARCHAR(20) UNIQUE,  
    columna3 INT PRIMARY KEY, -- solo válida si  
solo hay una columna en la clave primaria  
    columna4 DATE DEFAULT '2099-12-31',  
    columna5 VARCHAR(10) DEFAULT 'non',  
    columna6 INT DEFAULT 0,  
    columna7 CHECK (columna7>=10)  
)
```



SQL: Restricciones en creación de tablas

Las restricciones pueden escribirse en línea y en bloque. He aquí las más frecuentes:

- **NOT NULL**: Un campo no puede ser nulo.
- **UNIQUE**: El valor de un campo no puede repetirse.
- **PRIMARY KEY**: Este campo o campos forma(n) la clave primaria de la tabla.
- **FOREIGN KEY / REFERENCES**: Este campo es clave foránea y hace referencia a la tabla de la que viene.
- **DEFAULT**: Se especifica el valor por defecto que tomará este campo en caso de no quedar determinado.
- **CHECK**: Esta columna debe cumplir una condición concreta que tiene que ver con su valor.
- **CREATE INDEX**: Permite la creación de índices sobre la columna.

```
CREATE TABLE ejemplo (  
    columna1 VARCHAR(20),  
    ...  
  
    CONSTRAINT tabla_nn_columna NOT NULL (columna1),  
    CONSTRAINT tabla_uq_columna UNIQUE (columna2[,  
    columna3]) -- puede ser una combinación.  
    CONSTRAINT pk_tabla PRIMARY KEY (columna4, columna5)  
    -- si es más de una, tiene que ponerse aquí a la fuera.  
    CONSTRAINT fk_tabla1_columna_tabla2 FOREIGN KEY  
    (columna_ejemplo) REFERENCES otra_tabla(otra_columna)  
    CONSTRAINT ck_columna_tabla CHECK (columna1>10 OR  
    columna2 LIKE 'F%')  
)
```



SQL: Restricciones en creación de tablas

Las restricciones pueden escribirse en línea y en bloque. He aquí las más frecuentes:

- **NOT NULL:** Un campo no puede ser nulo.
- **UNIQUE:** El valor de un campo no puede repetirse.
- **PRIMARY KEY:** Este campo o campos forma(n) la clave primaria de la tabla.
- **FOREIGN KEY / REFERENCES:** Este campo es clave foránea y hace referencia a la tabla de la que viene.
- **DEFAULT:** Se especifica el valor por defecto que tomará este campo en caso de no quedar determinado.
- **CHECK:** Esta columna debe cumplir una condición concreta que tiene que ver con su valor.
- **CREATE INDEX:** Permite la creación de índices sobre la columna.

```
CREATE [UNIQUE] INDEX índice -- UNIQUE no permite duplicados  
ON tabla (columna_indexada_1, columna_indexada_2, ...);
```



SQL: CTAS (CREATE TABLE AS SELECT)

```
CREATE TABLE [IF NOT EXISTS] tabla_ctas AS (  
    SELECT c1, c2  
    FROM otra_tabla  
    WHERE condición  
)
```

Ejemplo de uso de subconsultas en la creación de tablas. Unifica la creación de tablas con la inclusión de datos en la misma usando una subconsulta.

Por lo demás, una tabla creada mediante CTAS funciona exactamente igual que una tabla normal.



ALTER TABLE tabla

-- ADD COLUMN columna TIPO_DATO

-- DROP COLUMN columna

-- RENAME COLUMN columna TO otra_columna

-- MODIFY COLUMN columna NUEVO_TIPO_DATO

-- DROP CONSTRAINT nombre_restriccion

-- ADD CONSTRAINT nombre_restriccion

CONFIG_CONSTRAINT (lo que ya hemos visto antes
para hacerlas)

Para alterar los distintos aspectos de una tabla, con las siguientes opciones:

- **ADD COLUMN:** Añade una columna.
- **DROP COLUMN:** Borra una columna.
- **RENAME COLUMN:** Renombra una columna (le cambia el nombre).
- **MODIFY COLUMN:** Modifica el tipo de dato de una columna (por ejemplo, para cambiar la longitud de un varchar).
- **DROP CONSTRAINT:** Elimina una restricción.
- **ADD CONSTRAINT:** Añade una restricción, correctamente configurada.

SQL: DROP TABLE y TRUNCATE TABLE

```
TRUNCATE TABLE tabla_ejemplo;
```

```
DROP TABLE tabla_ejemplo;
```

DROP TABLE borra la tabla y todos sus datos.

TRUNCATE TABLE solo borra todos los datos de la tabla, dejándola vacía.



SQL: RENAME TABLE

```
RENAME TABLE tabla TO otra_tabla  
[, tabla2 TO otra_tabla2  
, ...]
```

Sirve para cambiarle el nombre a una o varias tablas.



SQL: INSERT INTO

```
INSERT INTO tabla (columna1, columna2) -- si la tabla tiene más columnas han de poder ser NULL
VALUES ('Johnny', 135),
      ('Pepi', 111),
      ('El Otro', 8);
```

```
INSERT INTO tabla VALUES ('Juanito', 15, 'C1'), -- no es necesario incluir las columnas si se va a
insertar en todas las columnas.
      ('Pepón', 20, 'C2');
```

INSERT INTO funciona insertando uno o más datos en la tabla. Solo es necesario especificar las columnas si no se va a insertar en todas ellas a la vez. Si, como se ve en el ejemplo, se inserta en todas las columnas de la tabla, no es necesario especificar en cuáles se va a insertar.



SQL: INSERT INTO SELECT

```
INSERT INTO tabla (columna_varchar, columna_int)
SELECT otra_col_varchar, otra_col_int
FROM otra_tabla
WHERE condición
```

-- Sin seleccionar columnas (deben coincidir los esquemas)

```
INSERT INTO tabla
SELECT * FROM otra_tabla
WHERE condicion
```

INSERT INTO también funciona usando subconsultas. Hay que tener cuidado que las columnas en las que se insertar y de las que se recuperan de la consulta sean el mismo número y con los mismos tipos de datos.



SQL: DELETE y UPDATE

DELETE FROM tabla WHERE condición -- sin condición = TRUNCATE TABLE

UPDATE tabla

SET columna = ('valor'|subconsulta)

[, columna2 = 'otro',

...]

WHERE condición

DELETE FROM borra todos los registros de una tabla que cumplan una condición (si no se pone condición, se borran todos los registros, similar a TRUNCATE TABLE).

UPDATE sobrescribe el valor que tuviera la columna antes por el que se determine (ya sea mediante valor o subconsulta que devuelva un único valor) en todos los registros que cumplan la condición (si no se pone condición, se sobrescriben todos los registros).



SQL: Operaciones de ventana

```
SELECT duracion_segundos,  
       SUM (duracion_segundos) OVER (ORDER BY start_time) AS acumulado  
FROM carreras;
```

En el ejemplo vemos un total acumulado, una operación de ventana muy común.

Las **operaciones de ventana** son una poderosa y reciente herramienta en SQL. Realizan un cálculo sobre un conjunto de filas relacionadas con la fila actual. Parecido a una agrupación pero sin agrupar las filas, cada fila se mantiene separada.

Para dividir el cálculo del acumulado en grupos concretos, podemos usar PARTITION BY:

```
SELECT start_terminal,  
       SUM (duracion_segundos) OVER (PARTITION BY start_terminal ORDER BY start_time) AS  
running_total  
FROM carreras  
WHERE start_time < '2012-01-08';
```



OVER es la palabra clave para determinar que trabajamos con funciones de ventana.

ORDER BY y **PARTITION BY** definen la ventana (ORDER BY solo ordena dentro de cada partición si los datos están particionados): el subconjunto ordenado de datos sobre el que se hacen los cálculos.

Importante: No pueden usarse operaciones de ventana y agregaciones estándar en la misma consulta. Es decir, no pueden incluirse funciones de ventana en un GROUP BY.



- **SUM, COUNT, AVG:** Cualquier función de agregación puede servir como función de ventana.

- **ROW_NUMBER(), RANK() y DENSE_RANK():**

Muestran el número de orden de una fila concreta, empezando por 1. No necesita especificar una variable entre los paréntesis. Usando **PARTITION BY** implica que tras cada partición se reinicia la cuenta.

La diferencia entre estos es la siguiente: los empates.

RANK() asigna un ranking según el orden establecido, saltándose números: 1, 1, 3, 4, 4, 6.

DENSE_RANK() hace lo mismo pero sin saltarse números: 1, 1, 2, 3, 3, 4.

ROW_NUMBER() simplemente asigna los números sin tener en cuenta empates: 1, 2, 3, 4, 5, 6.

- **NTILE(n):** Identificación de cuartiles, percentiles, etc. Según el n que se ponga. Especificando la partición por la que se reinicia el conteo, funciona igual que los anteriores, añadiendo una columna a los resultados con el cuartil, quintil, percentil, etc. Al que pertenece. Divide el conjunto de resultados en n partes, básicamente.
- **LAG y LEAD:** Útil para comparar registros con valores anteriores o posteriores. Su sintaxis es **LAG/LEAD(campo, n)**, siendo n cuántos registros atrás o adelante contamos para comparar (2 sería el registro dos filas más allá, 3 tres filas más allá, etc.) Útil para operaciones como comparar ventas de un día para otro, o de un lunes a otro (7 sería el n en ese caso), y operaciones similares.



SQL: CTES – Common Table Expressions (WITH)

```
WITH identificador_bloque AS (  
-- Consulta completa y funcional  
)  
[, otro_identificador AS (  
-- Otra consulta  
)  
SELECT * FROM otro_identificador;
```

Herramienta muy útil para la realización de consultas complejas, ya que permite dividir el trabajo en bloques autocontenidos menos complejos usando la técnica de “divide y vencerás”, encadenando los distintos segmentos hasta llegar al resultado final de manera que hemos creado un código más legible.



SQL: CASE WHEN

```
SELECT columna1, CASE  
    WHEN condicion1 THEN resultado1  
    WHEN condicion2 THEN resultado2  
    WHEN condicion3 THEN resultado3  
    ELSE resultado_por_defecto  
END as columna_case  
FROM tabla;
```

La expresión **CASE WHEN** evalúa ciertas condiciones, devolviendo un valor a la primera condición que se resuelva con éxito (parecido a un if-then-else). Una vez encuentre una condición verdadera, deja de leer y devuelve el resultado. Por ello es importante el orden de los enunciados WHEN.



SQL: DCL y TCL

DCL: Lenguaje de control de permisos, básicamente, basado en dar (GRANT) y quitar (REVOKE) permisos):

GRANT permiso ON recurso TO usuario;

REVOKE permiso ON recurso FROM usuario;

Los permisos que se pueden asignar y quitar son los siguientes, consistentes en realizar operaciones con los siguientes comandos sobre los recursos asignados (normalmente, tablas): **SELECT**, **INSERT**, **UPDATE**, **DELETE**, **REFERENCES** (claves foráneas), **ALTER**, cualquier combinación de las anteriores, y **ALL** (todas las anteriores).

TCL: Control de transacciones. Refuerza lo que vimos en las propiedades ACID.

- **BEGIN / END:** Comienza una transacción / Termina una transacción.
- **COMMIT:** Guarda todo lo realizado desde el anterior commit / inicio de la transacción.
- **ROLLBACK:** Usado en caso de posible error, devuelve la base de estados a un estado anterior correspondiente al anterior commit o antes del comienzo de la transacción.



Listado de ejercicios de este módulo.

